# Számítógépes Hálózatok

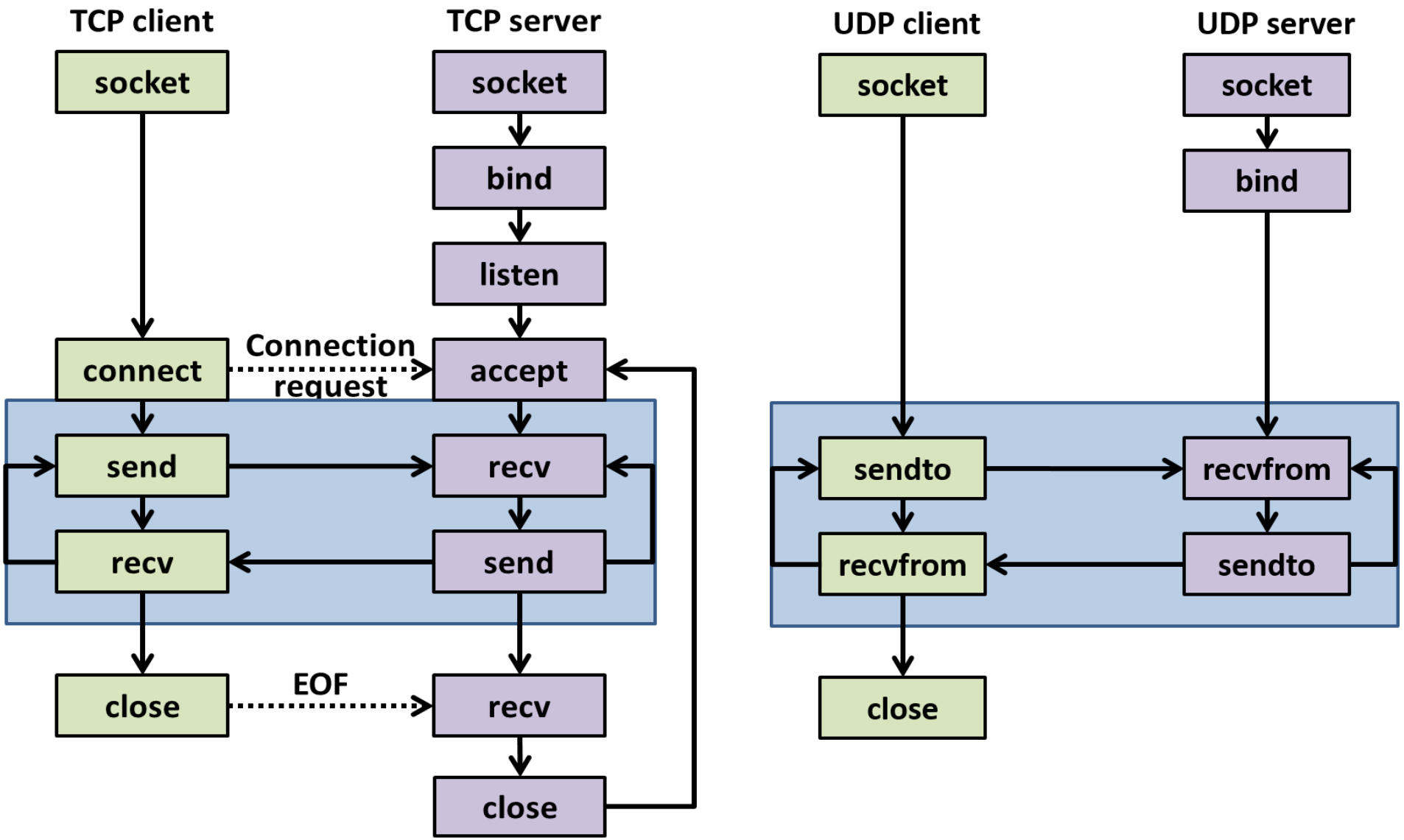
5. gyakorlat

# PYTHON SOCKET - UDP

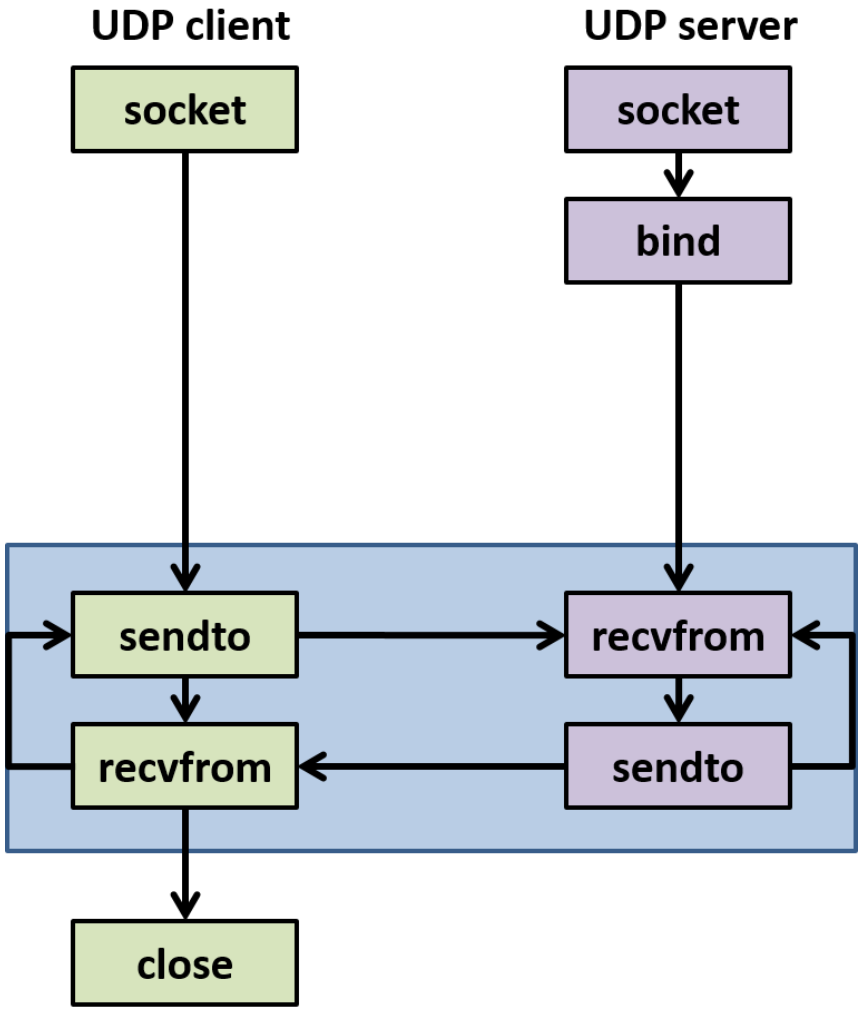
# A kommunikációs csatorna kétféle típusa

- Kapcsolat-orientált modell (analógia: telefonbeszélgetés)
  - csomagok megérkeznek jó sorrendben
  - ilyen protokoll a TCP
  - kapcsolódó típus: stream socket
- **Kapcsolat-nélküli modell (analógia: postai levelezés)**
  - **csomagok nem biztos, hogy sorrend helyesen érkeznek, sőt el is veszhetnek**
  - **előnye a jobb teljesítmény**
  - **ilyen protokoll a UDP**
  - **kapcsolódó típus: datagram socket**

# TCP VS UDP


TCP client
TCP server
socket
socket
bind
listen
connect
Connection request
accept
send
recv
recv
send
close
EOF
recv
close
UDP client
UDP server
socket
socket
bind
sendto
recvfrom
recvfrom
sendto
close

# UDP

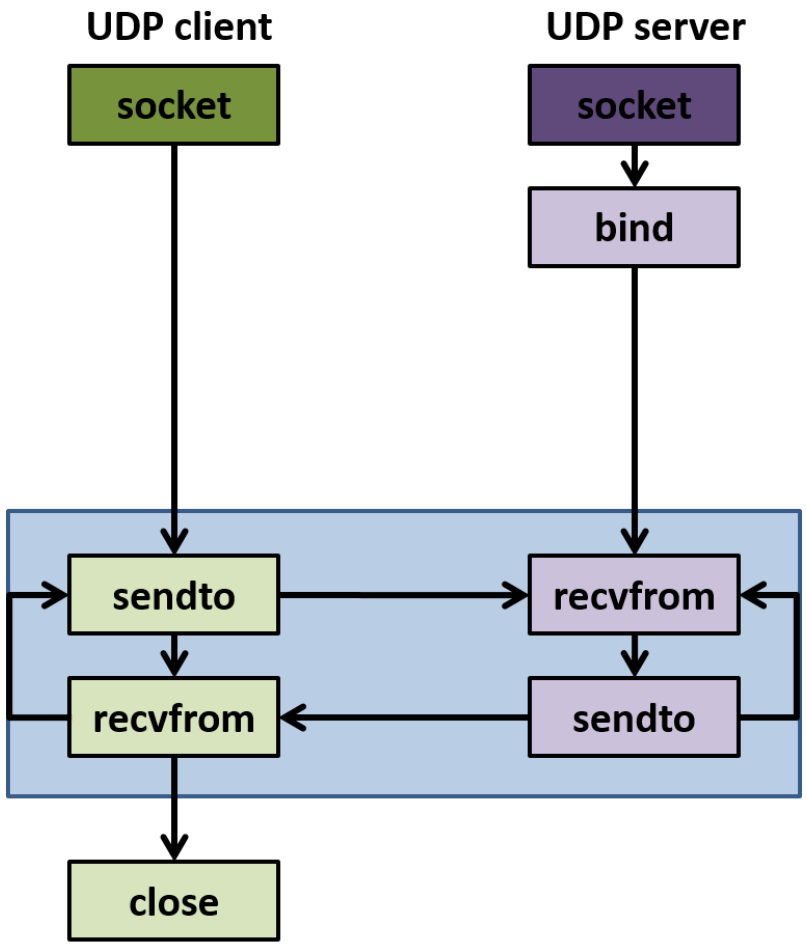

UDP client
UDP server
socket
socket
bind
sendto
recvfrom
recvfrom
sendto
close

# Socket leíró beállítása

- socket.**socket**( [*family*
  [, *type*
  [, *proto*]]])
- *family* : socket.AF_INET → IPv4 (AF_INET6 → IPv6)
- ***type* : socket.SOCK_DGRAM → UDP**
- *proto* : 0

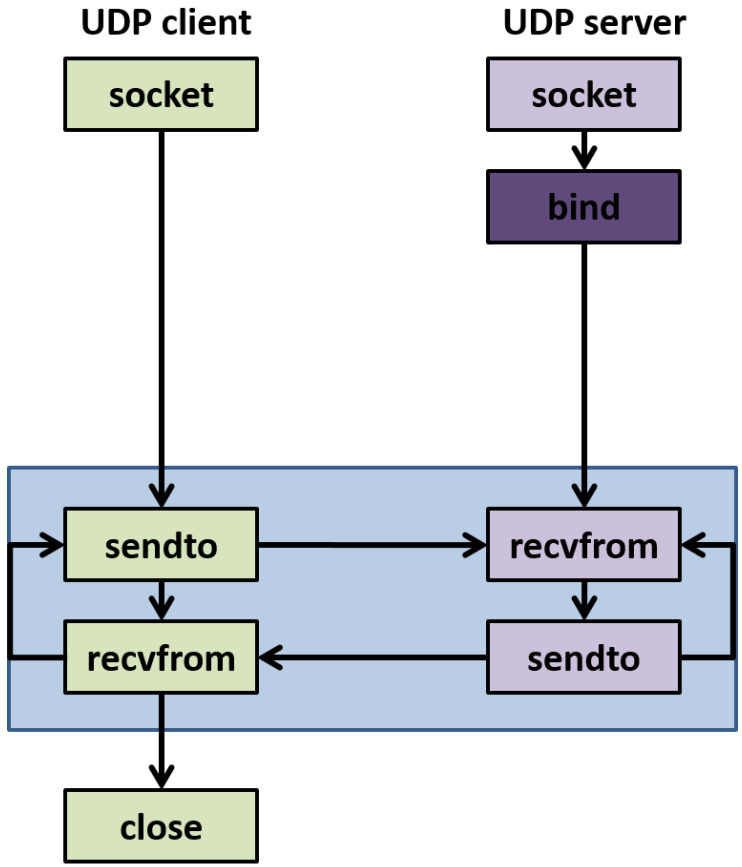
(alapértelmezett protokoll lesz)

- visszatérési érték: egy socket objektum, amelynek a metódusai a különböző socket rendszer hívásokat implementálják



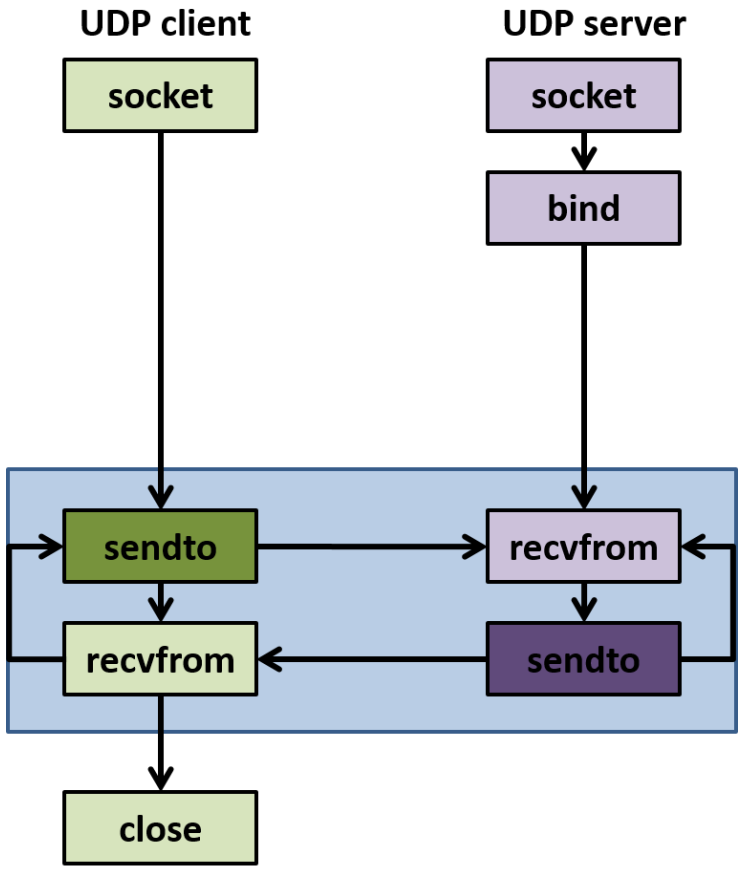
# Bindolás – ismétlés

- socket.socket.**bind**(*address*)
- A socket objektum metódusa
- *address* : egy tuple, amelynek az első eleme egy hosztnév vagy IP cím (sztring reprezentációval), második eleme a portszám



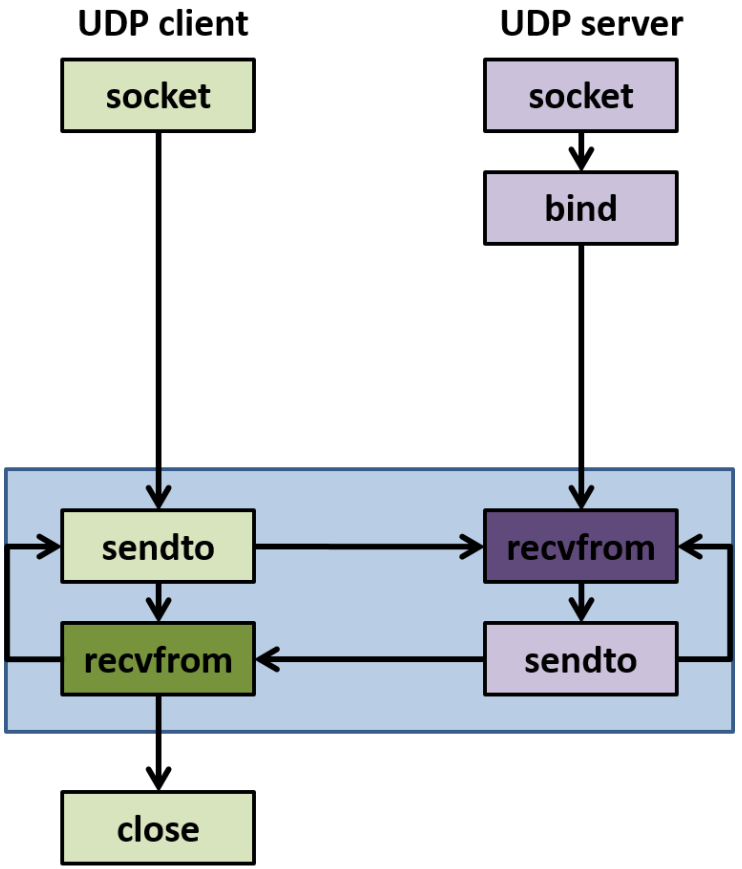
# sendto

- socket.socket.**sendto**(*string* , ***address***)
- socket.socket.**sendto**(*string* , *flags* , ***address***)
- A socket objektum metódusai
- Adatküldés (*string*) a socketnek
- *flags* : 0 (nincs flag meghatározva)
- **A socketnek előtte nem kell csatlakozni a távoli sockethez, mivel azt az *address* meghatározza**



# recvfrom

- socket.socket.**recvfrom**( *bufsize* [, *flags*])
- A socket objektum metódusa
- Üzenet fogadása
- *bufsize* : a max. adatmennyiség, amelyet egyszerre fogadni fog
- *flags* : 0 (nincs flag meghatározva)
- **visszatérési érték: egy (*string*, *address*) tuple, ahol a fogadott adat sztring reprezentációja és az adatküldő socket címe szerepel**



# UDP

- socket

```
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
```

- recvfrom()

```
data, address = sock.recvfrom(4096)
```

- sendto()

```
sent = sock.sendto(data, address)
```

# Feladat: Hello UDP felett

Készítsünk egy kliens-szerver alkalmazást, amely UDP protokollt használ. A kliens küldje a ‚Hello Server' üzenetet a szervernek, amely válaszolja a ‚Hello Kliens' üzenetet.

# Feladat - Számológép UDP felett

Készítsünk egy szerver-kliens alkalmazást, ahol a kliens elküld 2 számot és egy operátort a szervernek, amely kiszámolja és visszaküldi az eredményt. A kliens üzenete legyen struktúra. Használjunk UDP protokollt!

# ÓRAI FELADAT

# Órai Feladat – Számológép több UDP

- (1) Alakítsuk át a kliens működését úgy, hogy ne csak egy kérést küldjön a szervernek, hanem 10 kérés-válasz üzenetváltás történjen.
- Alakítsuk át hogy ne standard inputról, hanem random értékeket küldjön (random.randint(1,100)) structban!
- A szerver ugyan abban a struct formátumba küldje vissza az értéket, ahol az első szám lesz az eredmény, a többi adat pedig nem lényeges.
- (2) Ha kész, akkor egészítsük ki a programot, hogy a szerver mentse fájlba (txt vagy json) kérdéseket és válaszokat és hogy ki kérdezte.

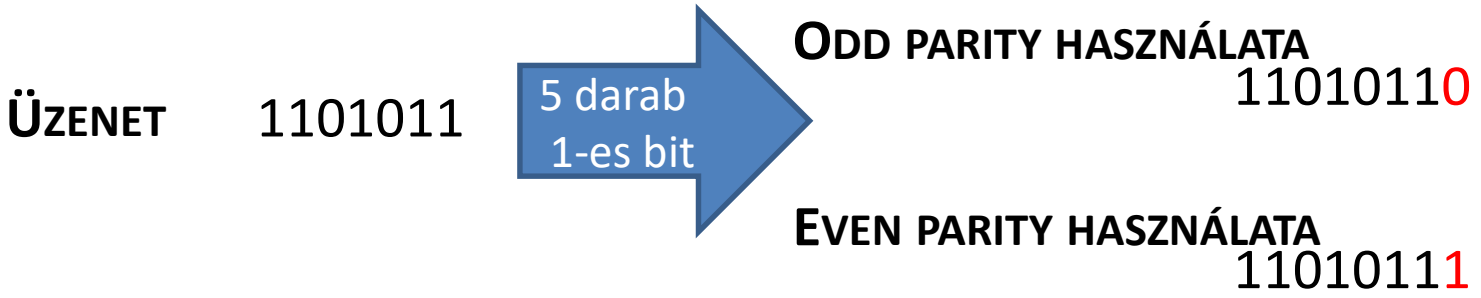
# REDUNDANCIA

# Redundancia

- Redundancia nélkül:
  - $2^m$ lehetséges üzenet írható le *m* biten
  - Ekkor minden hiba egy új helyes üzenetet eredményez → a hiba felismerése lehetetlen
- Emiatt egy keret felépítése:
  - *m* adat bit (üzenet bit)
  - *r* redundáns/ellenőrző bit (üzenetből számolt, új információt nem hordoz)
  - A teljes küldendő keret (kódszó) hossza: *n = m+r*.

# Paritás bit használata

- A paritásbitet úgy választjuk meg, hogy ha a kódszóban levő 1-ek száma
  - **Odd parity** – páratlan, akkor 0 befűzése; egyébként 1-es befűzése
  - **Even parity** – páros, akkor 0 befűzése; egyébként 1-es befűzése



# Hiba felügyelet Hamming távolsággal

- Hamming távolság: két azonos hosszúságú bitszóban a különböző bitek száma.
- Kiterjesztése azonos hosszúságú bitszavak $S$ halmazára:

$$d(S) \coloneqq \min_{x,y \in S \wedge x \neq y} d(x,y)$$

  - ($S$ halmazt hívják kódkönyvnek vagy egyszerűen kódnak is.)
- *d* bit **hiba felismeréséhez** a megengedett (helyes) keretek halmazában legalább *d+1* Hamming távolság szükséges.
- *d* bit **hiba javításához** a megengedett (helyes) keretek halmazában legalább *2d+1* Hamming távolság szükséges
- Egy $S \subseteq \{0,1\}^n$ **kód rátája** $R_S = \frac{\log_2|S|}{n}$.
  - (a hatékonyságot karakterizálja)
- Egy $S \subseteq \{0,1\}^n$ **kód távolsága** $\delta_S = \frac{d(S)}{n}$.
  - (a hibakezelési lehetőségeket karakterizálja)

# Feladat

- Adott $S$ kódkönyv: $S = [1000010,0011011,1011010,0011101]$
- Adjuk meg $S$ Hamming távolságát ($d(S)$)!
- Adjuk meg $S$ kód rátáját ($R_S$) és távolságát ($\delta_S$)!
- Mit mondhatunk $S$ hibafelismerő és javító képességéről? Igazoljuk az állításunkat!

# Megoldás

| | 1000010 | 0011011 | 1011010 | 0011101 |
|---|---|---|---|---|
| 1000010 | 0 | 4 | 2 | 6 |
| 0011011 | 4 | 0 | 2 | 2 |
| 1011010 | 2 | 2 | 0 | 4 |
| 0011101 | 6 | 2 | 4 | 0 |

$\rightarrow d(S) = 2$

- $R_S = \frac{\log_2 |S|}{n} = \frac{\log_2 4}{7} = 0.2857$ és $\delta_S = \frac{2}{7} = 0.2857$
- Max. 1 bithiba ismerhető fel, de 0 javítható (mivel a $d(S) = 2$)

# Feladat

Egyetlen paritásbit által nyújtottnál nagyobb biztonságot akarunk elérni, így olyan hibaészlelő sémát alkalmazunk, amelyben két paritásbit van: az egyik a páros, a másik a páratlan bitek ellenőrzésére.

- Mekkora e kód Hamming-távolsága?
- Mennyi egyszerű és milyen hosszú burst-ös hibát képes kezelni?

# Megoldás

- A kód Hamming-távolsága 2, mivel a páros pozíciókban lévő paritás bit független a páratlan pozíciókban levőtől, külön-külön pedig könnyen látszik, hogy a H-táv 2 → 1 hibát tudunk jelezni.
- A burst-ös hibánál 3 hosszúságúnál még épp tudjuk jelezni, ha baj van, mivel így vagy a páros vagy a páratlan pozíciókra csak 1 hiba fog esni, azt pedig jelezni fogja a megfelelő paritás bit.

# VÉGE